

School of Information Science and Engineering

International Crash Course and Practical Training Week

Lanzhou University, Gansu, China – Mon-Friday, July 20-24, 2026

Network and OS Foundations for Safe Robotic Actuation: *Latency, Reliability, and Event-Driven Control in Industrial Hair-Cutting Robots*

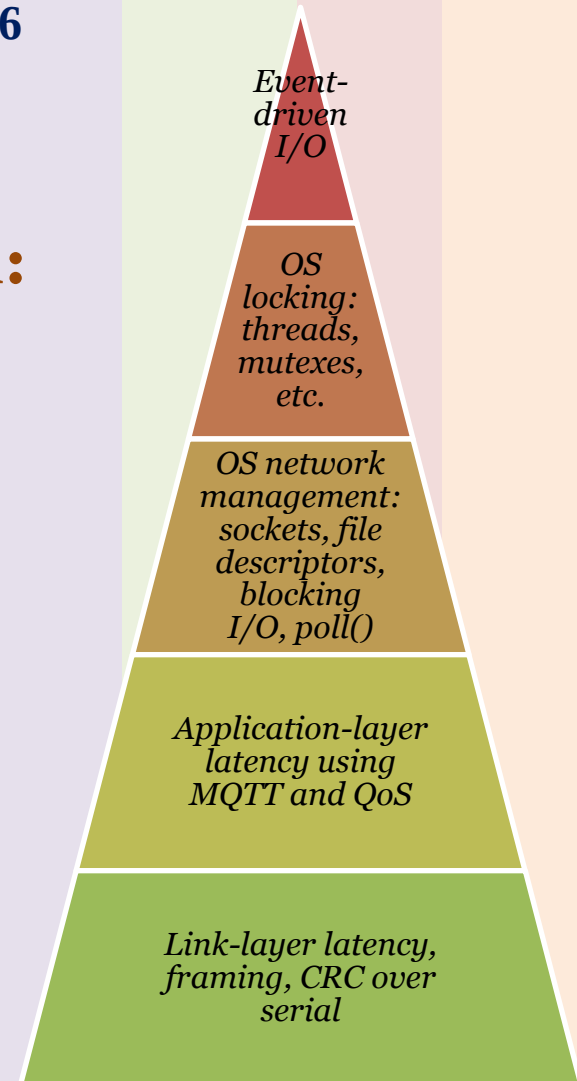
Audience Level: Advanced Undergraduate / Early Post-Graduate

Prepared and Presented by

Prof. Dr. Mohammed Aquil Mirza

*Fellow IESE (UK), Fellow IETE (India), Fellow HKIE (Hong Kong SAR),
Fellow AAIA (Singapore), Senior Member IEEE (USA), Life Member ACM (USA)*

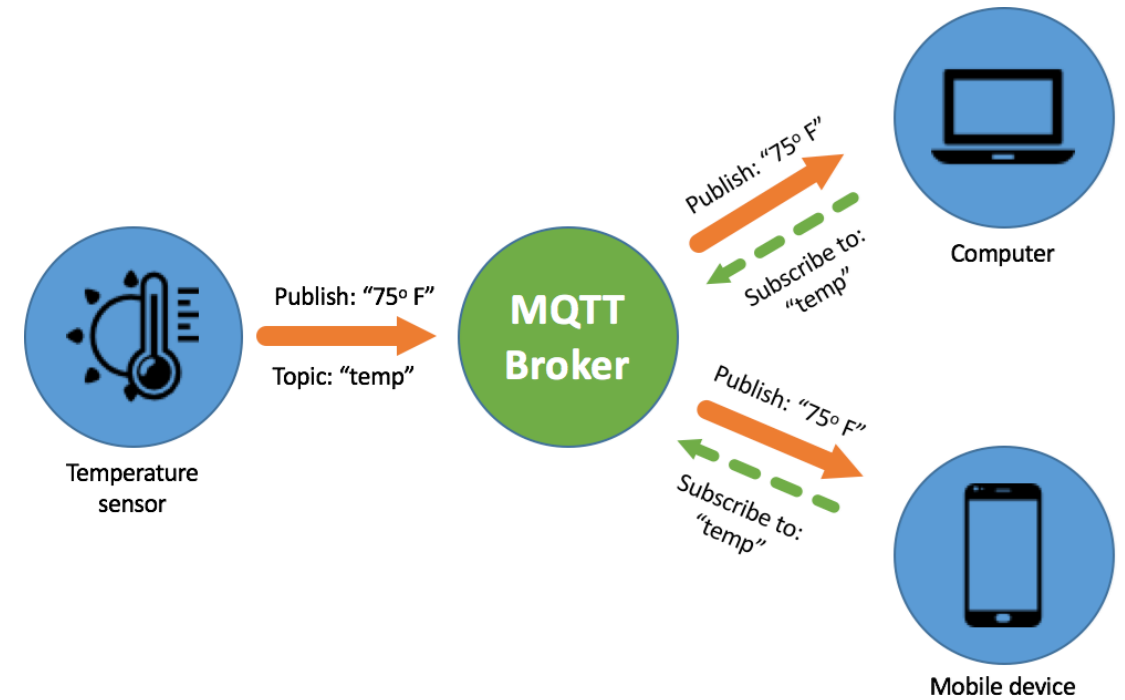
*Dean & Professor, School of Low Altitude and Digital Intelligence, Liaoning Technical University (LNTU), China
Senior Lecturer, Department of Computing, The Hong Kong Polytechnic University (PolyU), Hong Kong S.A.R.*



Introduction to MQTT

MQTT (Message Queuing Telemetry Transport) is a lightweight messaging protocol that enables efficient communication between IoT devices and applications, allowing real-time data exchange over various networks.

Low Cost, Low Power, and works with Unstable Connection



- Oil and pipe gas lines are located in remote areas.
- Challenging to get data from instruments (due to remote location and network bandwidth issues).
- Unavailability of data on time, late and bad response (due to bandwidth limitations)
- Options to access the data (Serial communication, Satellites, Radio Wave Communication (low bandwidth radio waves).



Introduction to MQTT

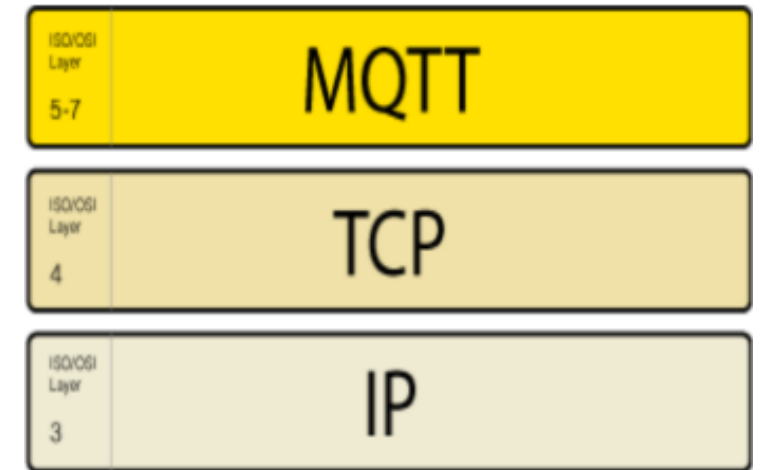
- MQTT developed in 1999 is an open standard with a short and readable protocol specification.
- Works on top of the TCP protocol stack
- October 29th 2014: MQTT was officially approved as OASIS Standard.
- Port 1883 is commonly used for unsecured MQTT
- MQTT 5.0 is the current version of the protocol.

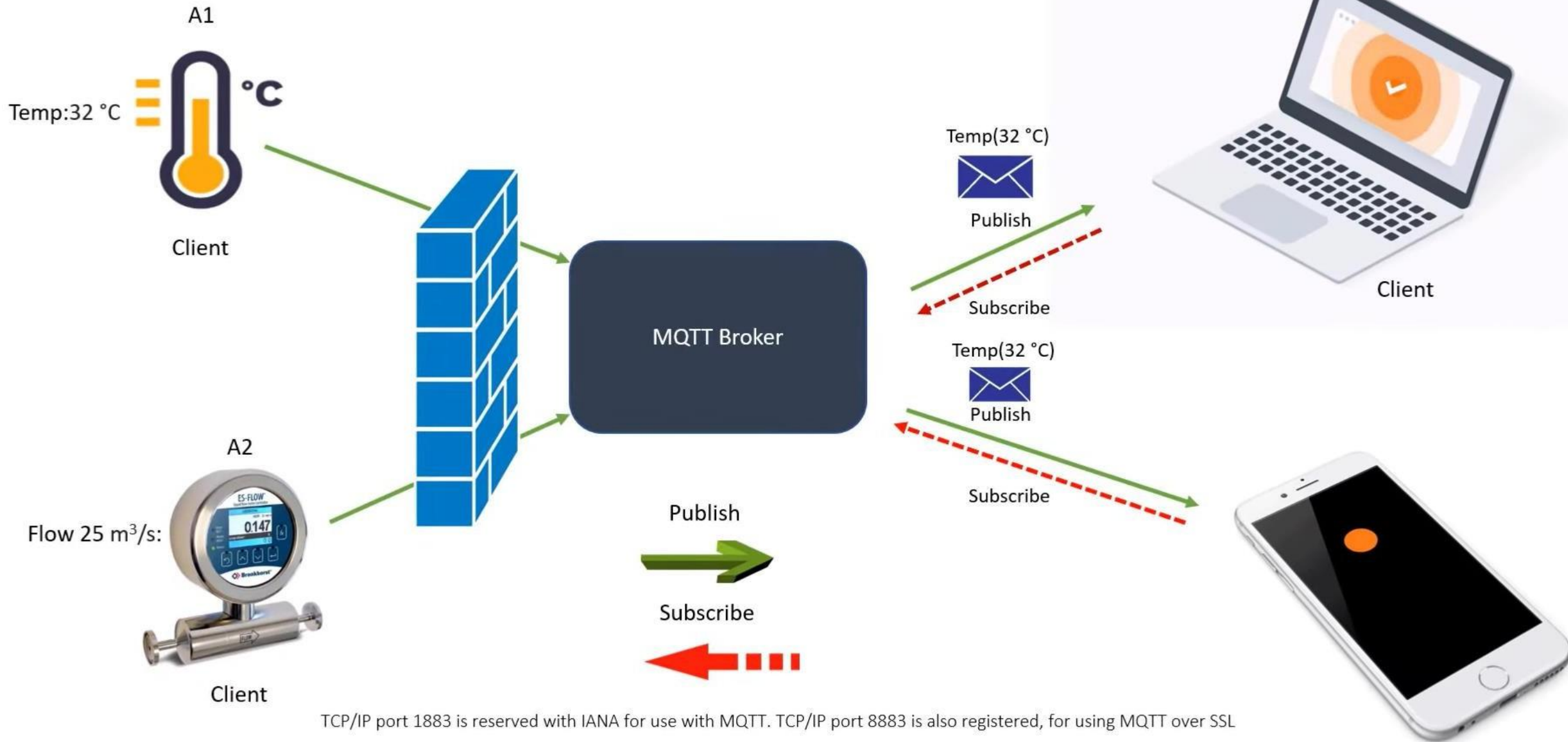
Components:

- Broker
- Publisher
- Subscriber

Methods:

- Connect
- Disconnect





TCP/IP port 1883 is reserved with IANA for use with MQTT. TCP/IP port 8883 is also registered, for using MQTT over SSL



< YouTube^{HK}    



Dr. Ir. Mohammed Aquil-Mirza
@AquilMirzaMohammed
17 subscribers • 10 videos

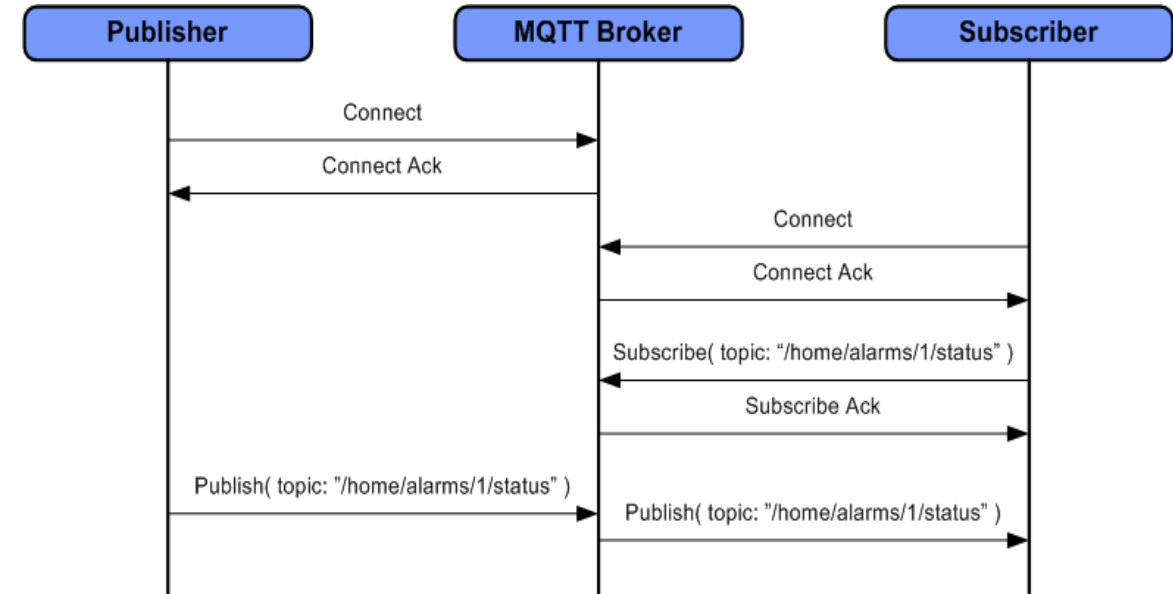
More about this channel ...more

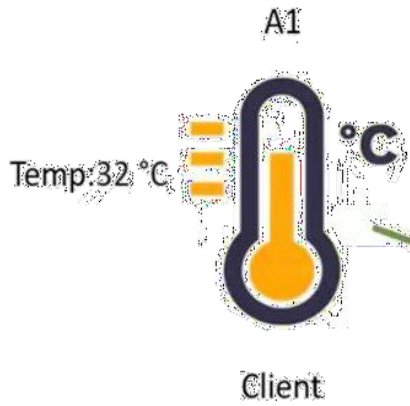
Subscribe

Latest Popular Oldest

MQTT “Publish” “Subscribe” Architecture

- The MQTT messages are delivered asynchronously (“push”) through publish subscribe architecture.
- The MQTT protocol works by exchanging a series of MQTT control packets in a defined way.
- Each control packet has a specific purpose and every bit in the packet is carefully crafted to reduce the data transmitted over the network.
- A MQTT topology has a MQTT server and a MQTT client.
- MQTT client and server communicate through different control packets. Table below briefly describes each of these control packets.



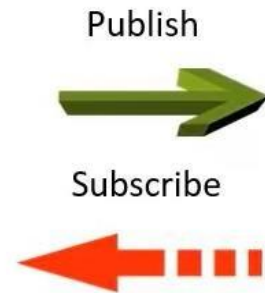
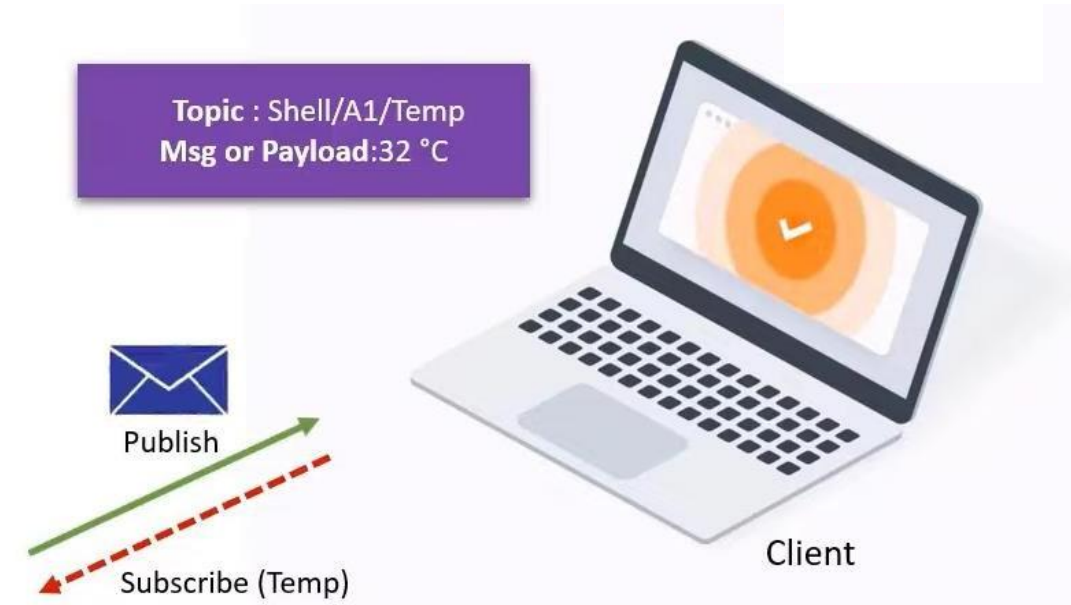
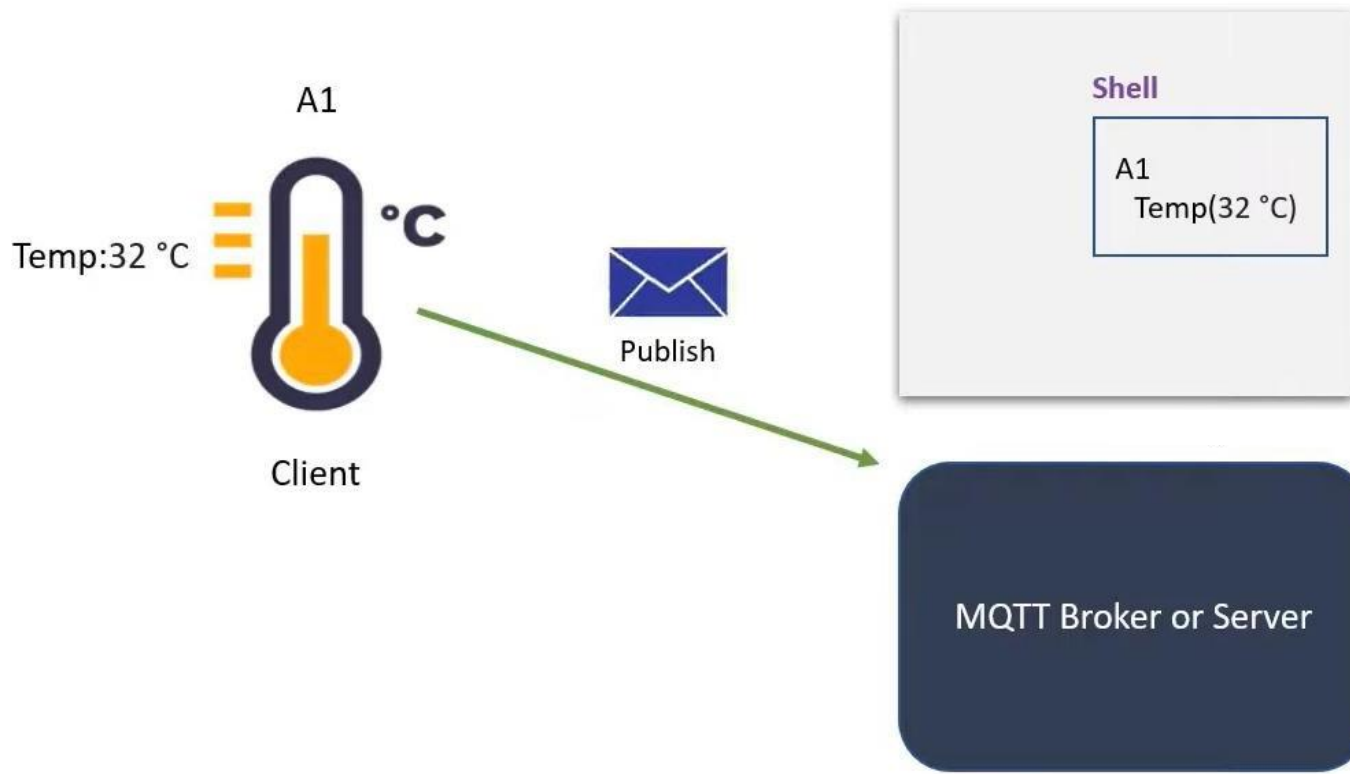


Topic : Shell/A1/Temp
Msg or Payload: 32 °C

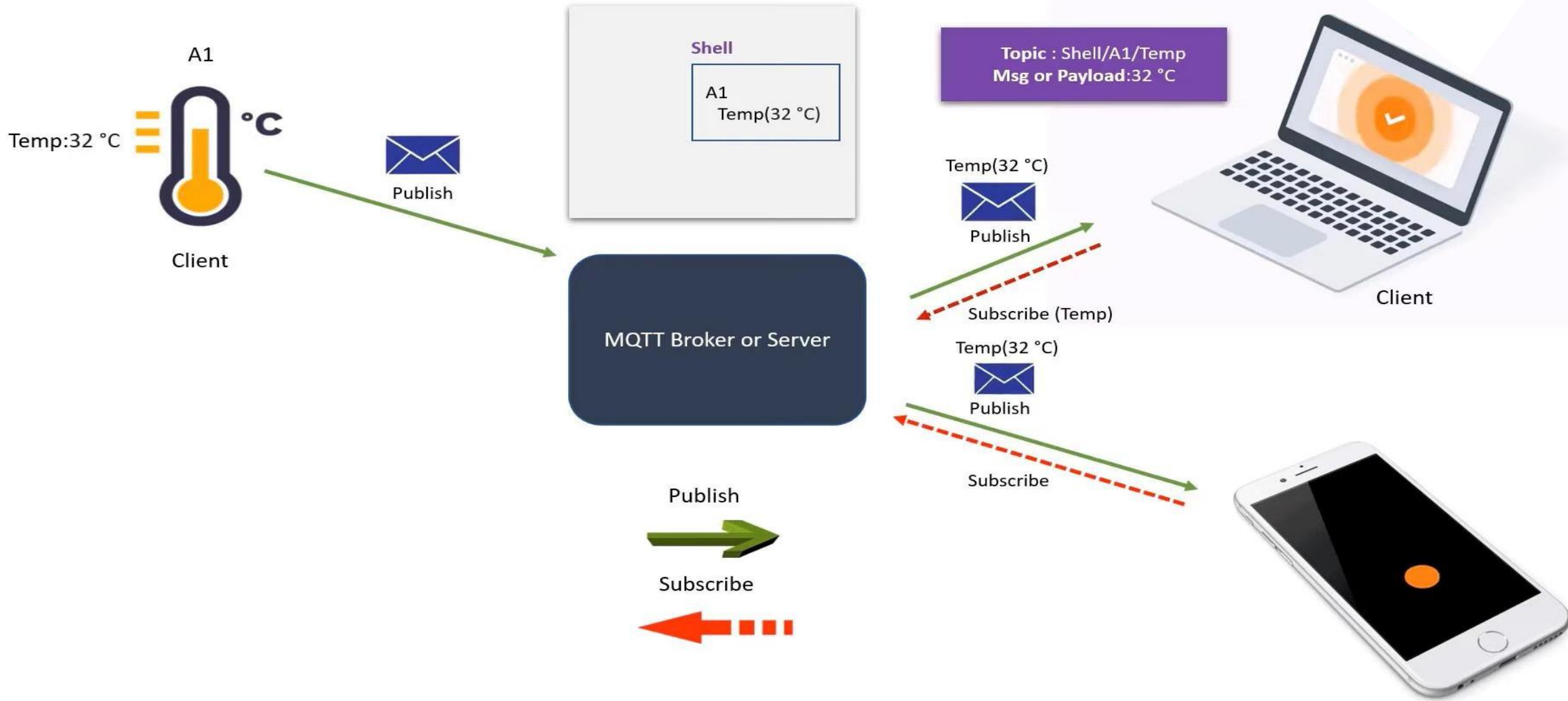


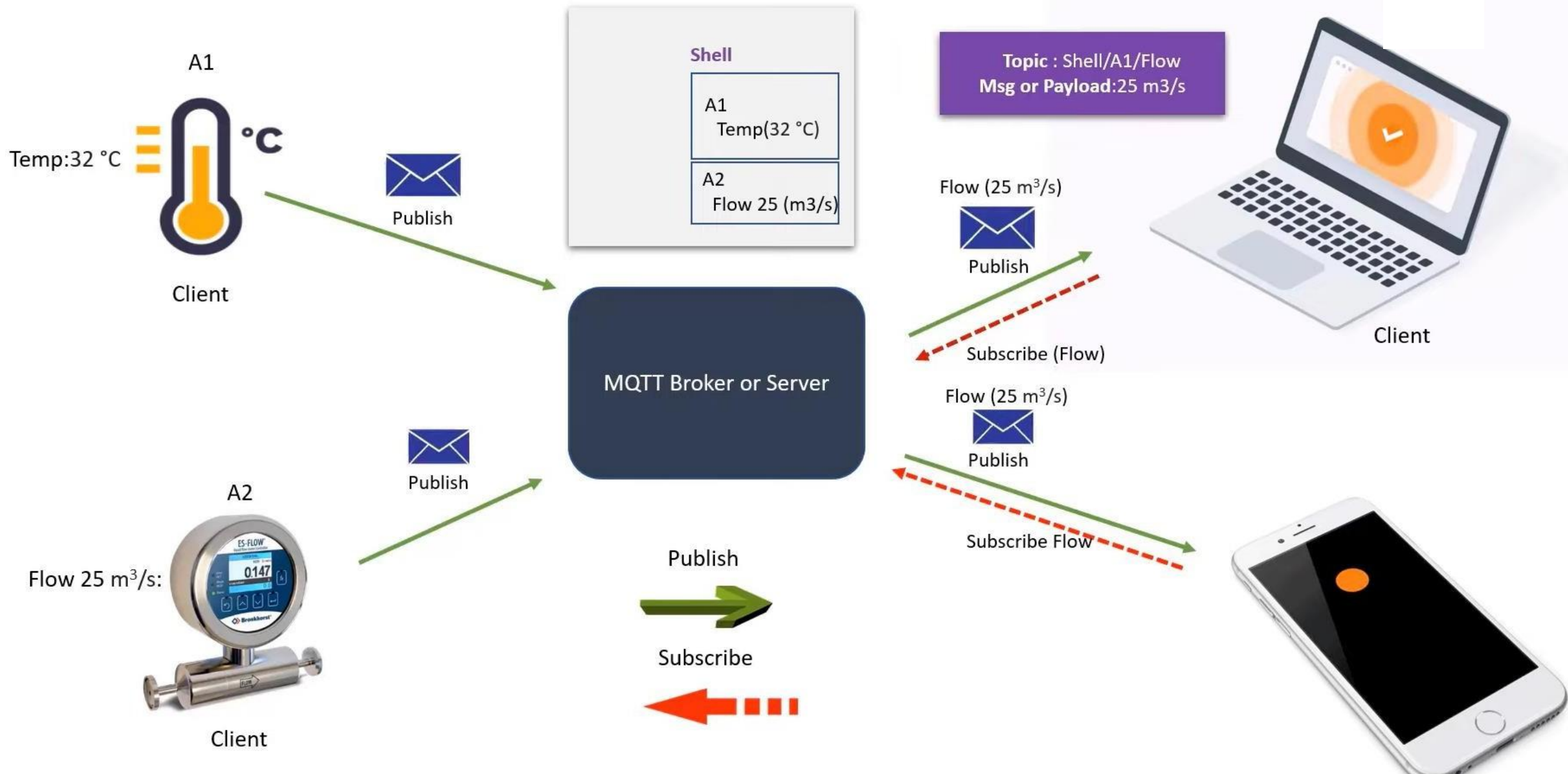
MQTT Broker or **server** acts as a heart of an MQTT.

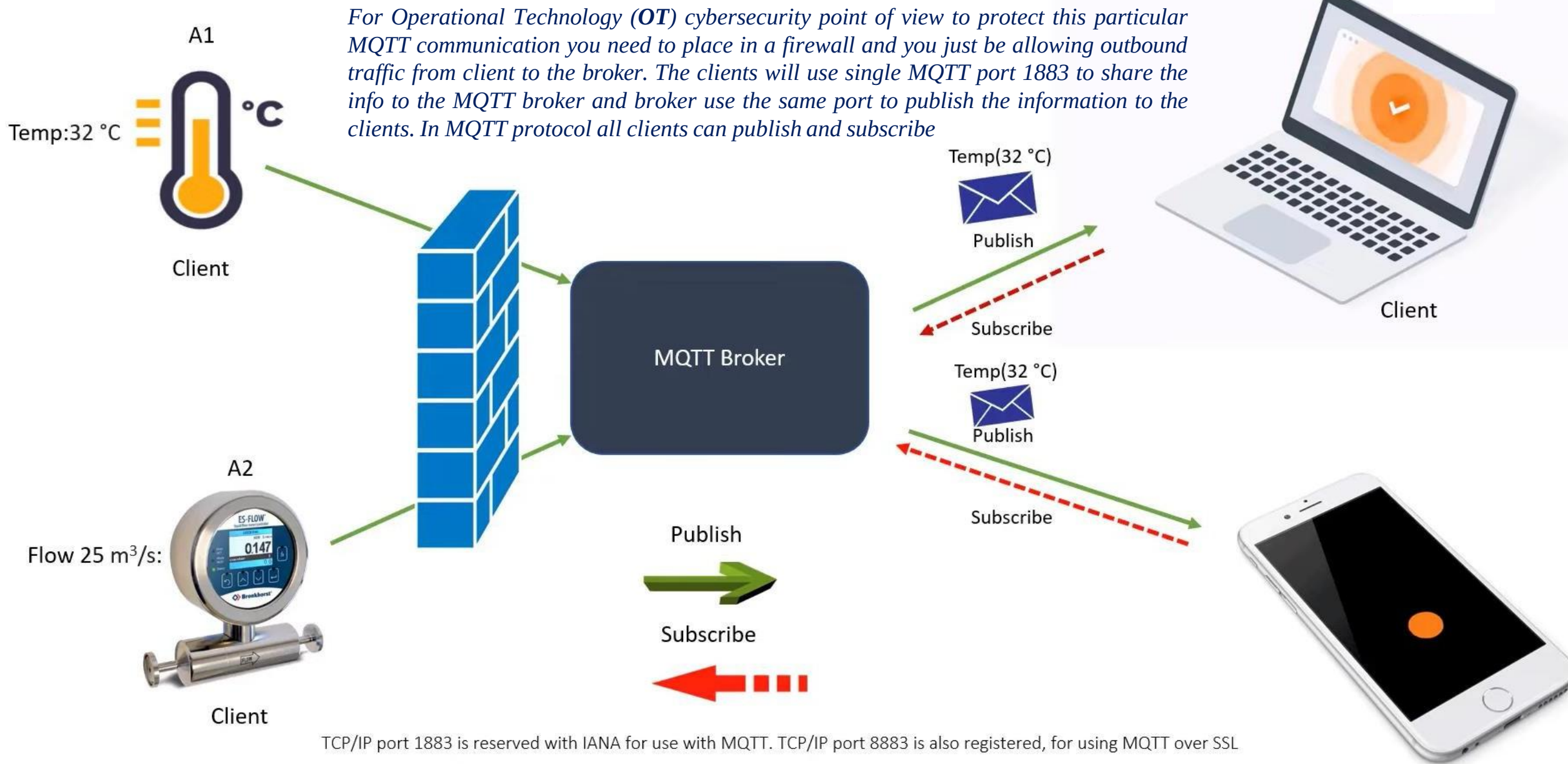
The broker is responsible for receiving all the messages and filter these messages, decides who is interested in them and then **publish** these messages to all **subscribers**



TCP/IP Port 1883 is reserved with IANA for use with MQTT and this port is also registered for using MQTT over SSL.



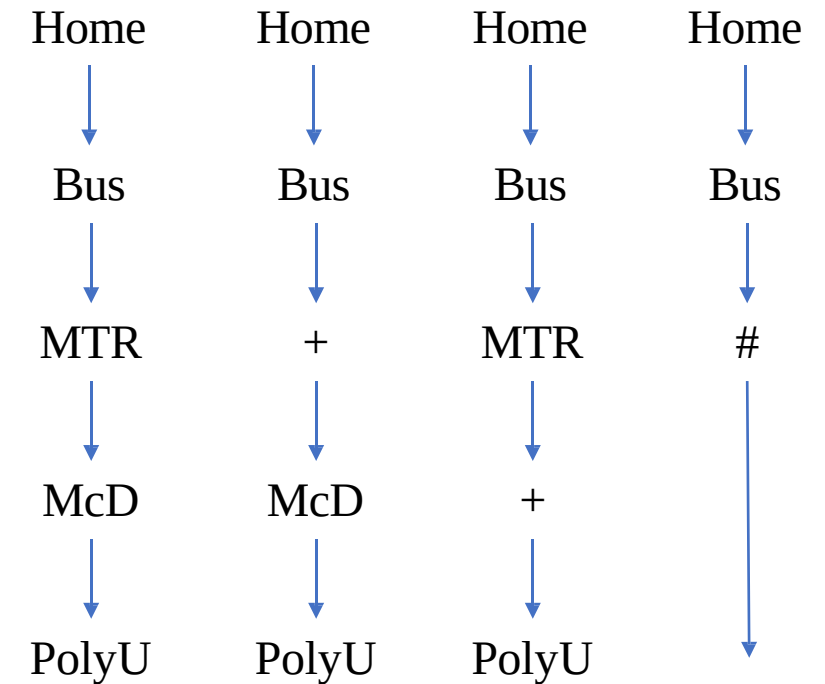




TCP/IP port 1883 is reserved with IANA for use with MQTT. TCP/IP port 8883 is also registered, for using MQTT over SSL

MQTT Topic and Wildcards

- **Topics are hierarchical (like filesystem path):**
 - */wsn/sensor/R1/temperature*
 - */wsn/sensor/R1/pressure*
 - */wsn/sensor/R2/temperature*
 - */wsn/sensor/R2/pressure*
- **A Subscriber can use wildcards in topics:**
 - */wsn/sensor/+/temperature*
 - */wsn/sensor/R1/+*
 - */wsn/sensor/#*



MQTT Topic: Details

A topic forms the namespace

- Is hierarchical with each “sub topic” separated by a /
- An example topic space
 - A house publishes information about itself on:
 - `<country>/<region>/<town>/<postcode>/<house>/energyConsumption`
 - `<country>/<region>/<town>/<postcode>/<house>/solarEnergy`
 - `<country>/<region>/<town>/<postcode>/<house>/alarmState`
 - `<country>/<region>/<town>/<postcode>/<house>/alarmState`
 - *And subscribes for control commands:*
 - `<country>/<region>/<town>/<postcode>/<house>/thermostat/setTemp`

A subscriber can subscribe to an absolute topic or can use wildcards:

- Single-level wildcards “+” can appear anywhere in the topic string
- Multi-level wildcards “#” must appear at the end of the string
- Wildcards must be next to a separator
- Cannot be used wildcards when publishing

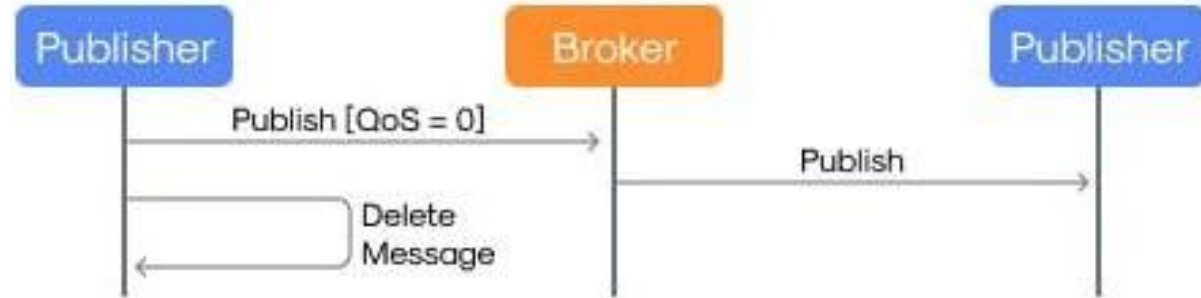
For example

- `UK/Hants/Hursley/SO212JN/1/energyConsumption`
- Energy consumption for 1 house in Hursley:
- `UK/Hants/Hursley/+ /+ /energyConsumption`
- Energy consumption for all houses in Hursley
- `UK/Hants/Hursley/SO212JN/#`
- Details of energy consumption, solar and alarm for all houses in SO212JN

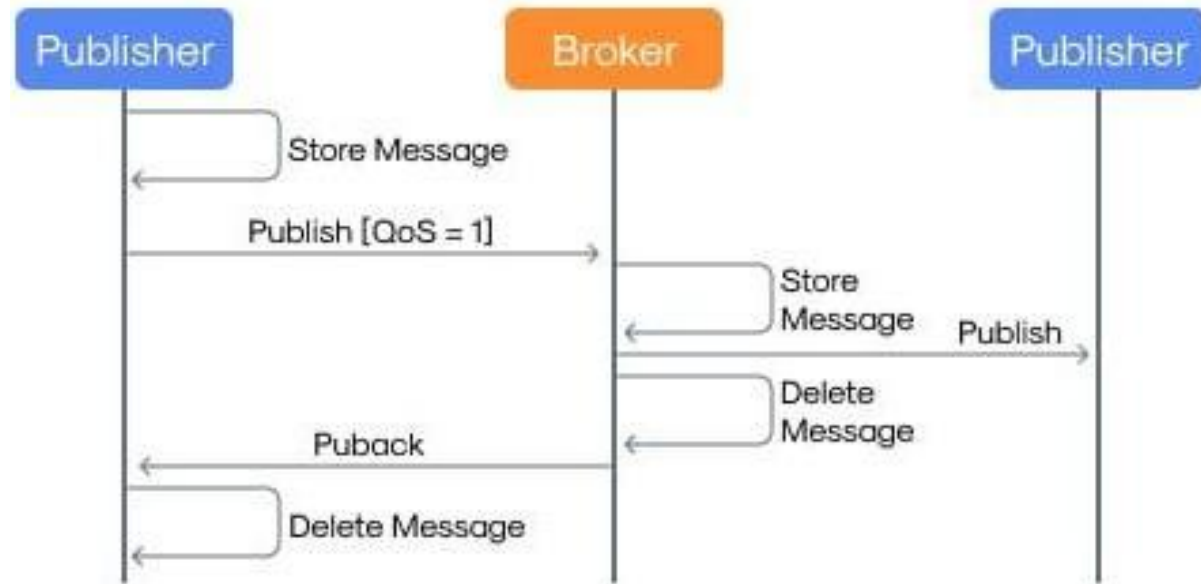
Quality of Service (QoS) for MQTT

- Quality of service (QoS) levels determine how each MQTT message is delivered and must be specified for every message sent through MQTT.
 - Three QoS for message delivery could be achieved using MQTT:
 - **QoS 0** (At most once) - where messages are delivered according to the best efforts of the operating environment. Message loss can occur.
 - **QoS 1** (At least once) - where messages are assured to arrive but duplicates can occur.
 - **QoS 2** (Exactly once) - where message are assured to arrive exactly once.
 - There is a simple rule when considering performance impact of QoS : “The higher the QoS, the lower the performance”.
-
- **PUBACK:** The publisher sends a PUBACK to acknowledge the receipt of the message.
 - **PUBREC:** The broker responds with a PUBREC to acknowledge the message.
 - **PUBREL:** The publisher sends a PUBREL to allow the broker to release the message.
 - **PUBCOMP:** The broker sends a PUBCOMP to confirm the completion of the message delivery.

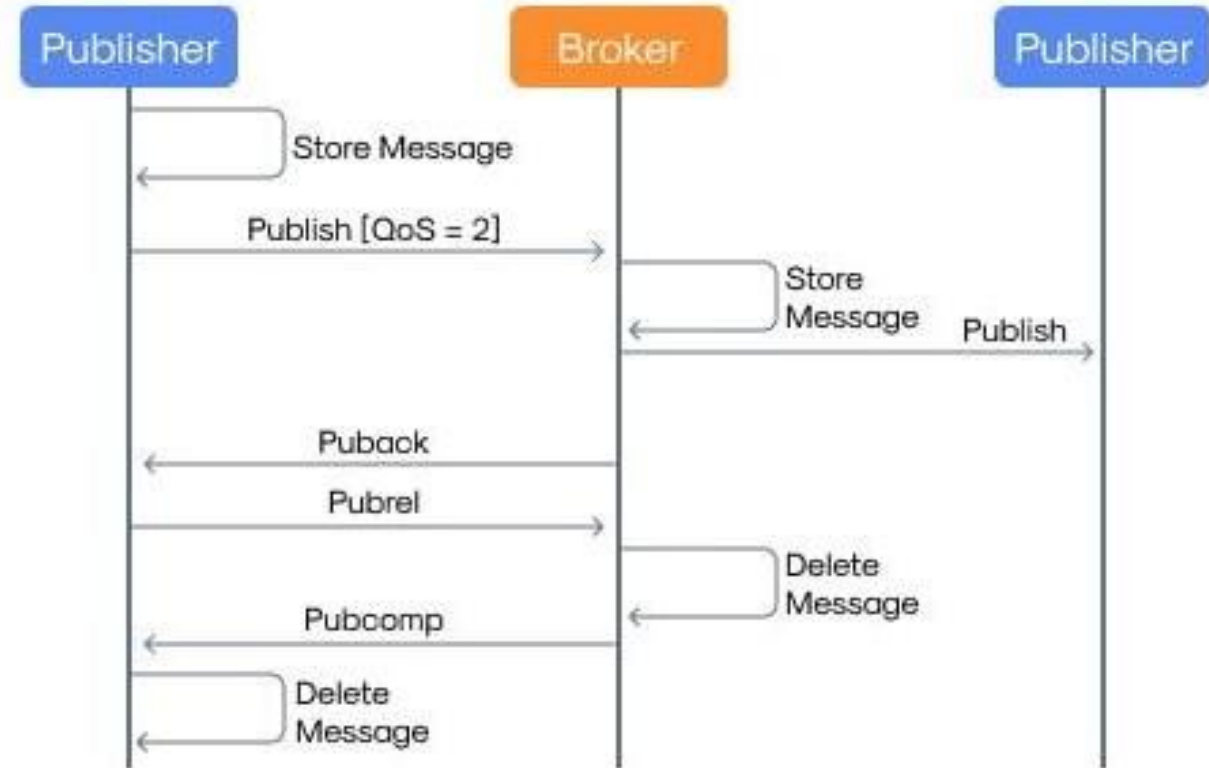
QoS 0: At Most Once (Fire And Forget)



QoS 1: At Least Once



QoS 2: Exactly Once



Security

- MQTT provides security, but it is not enabled by default.
 - As a basic solution we can rely on the encrypted WiFi connection to provide a basic level of security.
- MQTT has the option for Transport Layer Security (TLS) encryption.
- MQTT also provides username/password authentication.
 - Note that the password is transmitted in clear text. Thus, be sure to use TLS encryption if you are using authentication.

Software available

- **Brokers:**
 - <http://mosquitto.org/>
 - <http://www.hivemq.com/>
 - <https://www.rabbitmq.com/mqtt.html>
 - <http://activemq.apache.org/mqtt.html>
 - <https://github.com/mcollina/mosca>
- **Clients**
 - <http://www.eclipse.org/paho/>
 - Source: <https://github.com/eclipse/paho.mqtt.python>
 - Documentation: <https://pypi.python.org/pypi/paho-mqtt>
 - More docs: <http://www.eclipse.org/paho/clients/python/docs/>
 - <http://www.hivemq.com/demos/websocket-client/>
- **Tools**
 - <https://github.com/mqtt/mqtt.github.io/wiki/tools>
 - For example:
 - <http://www.hivemq.com/demos/websocket-client/>

MQTT (Message Queuing Telemetry Transport)

- **Purpose:** MQTT is designed for devices to communicate with each other, especially in the Internet of Things (IoT).
- **How It Works:** Instead of sending messages directly to each device, MQTT uses a central server called a **broker**. Devices (clients) send messages to the broker, which then **distributes** those messages to other **interested devices**.
- **Efficiency:** MQTT is lightweight and uses less data, making it great for devices with limited resources, like sensors or smart home devices.
- **Real-Time Communication:** It allows for real-time updates, so if one device sends a message, other devices can receive it almost instantly.

HTTP (Hypertext Transfer Protocol)

- **Purpose:** HTTP is the foundation of data communication on the web. It's used for loading web pages and transferring data between clients (like your web browser) and servers.
 - **How It Works:** When you request a web page, your browser sends an HTTP request to a server. The server then responds with the requested information, which your browser displays.
 - **Request-Response Model:** HTTP works on a **request-response basis**, meaning a **client must always ask for data**, and the server responds. There's no continuous connection like in MQTT.
 - **More Data Intensive:** HTTP can be heavier and uses more data compared to MQTT, which might not be ideal for low-power devices.
- **MQTT** is great for real-time communication between devices, especially in IoT, using a lightweight model with a broker.
 - **HTTP** is used for web pages and data transfer on the internet, relying on a request-response system.

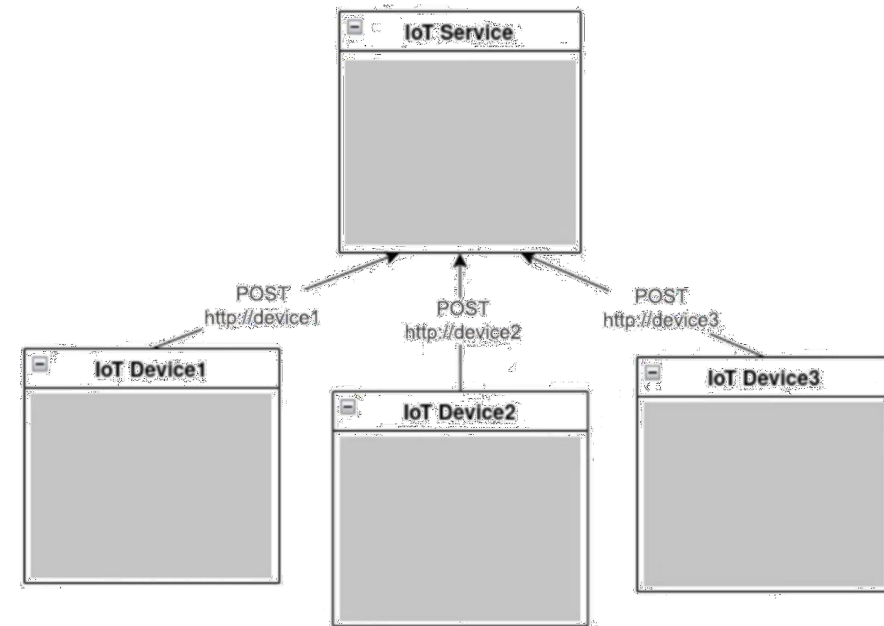
MQTT

HTTP

Architecture	Publish subscribe (MQTT does have a request/reply mode as well)	Request response
Command targets	Topics	URIs
Underlying Protocol	TCP/IP	TCP/IP
Secure connections	TLS + username/password (SASL support possible)	TLS + username/password (SASL support possible)
Client observability	Known connection status (will messages)	Unknown connection status
Messaging Mode	Asynchronous, event-based	Synchronous
Message queuing	The broker can queue messages for disconnected subscribers	Application needs to implement
Message overhead	2 bytes minimum. Header data can be binary	8 bytes minimum (header data is text - compression possible)
Message Size	256MB maximum	No limit but 256MB is beyond normal use cases anyway.
Message distribution	One to many	One to one
Reliability	Three qualities of service: 0 - fire and forget, 1 - at least once, 2 - once and only once	Has to be implemented in the application

MQTT vs HTTP

- **HTTP** stands for Hypertext Transfer Protocol and was created to make documents available across the internet.
- Think of **MQTT** as a quick chat tool for devices, while **HTTP** is like sending letters back and forth to get information!

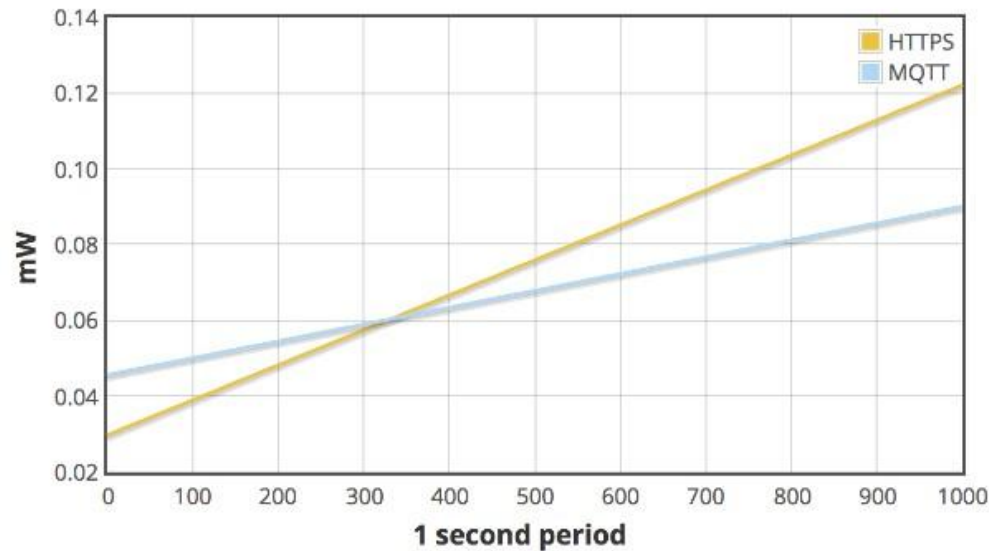


Energy usage: some number

amount of power taken to establish the initial connection to the server:

% Battery Used			
3G		Wifi	
HTTPS	MQTT	HTTPS	MQTT
0.02972	0.04563	0.00228	0.00276

3G – 240s Keep Alive – % Battery Used Creating and Maintaining a Connection



cost of 'maintaining' that connection (in % Battery / Hour):

	% Battery / Hour			
	3G		Wifi	
Keep Alive (Seconds)	HTTPS	MQTT	HTTPS	MQTT
60	1.11553	0.72465	0.15839	0.01055
120	0.48697	0.32041	0.08774	0.00478
240	0.33277	0.16027	0.02897	0.00230
480	0.08263	0.07991	0.00824	0.00112

you'd save ~4.1% battery per day just by using MQTT over HTTPS to maintain an open stable connection.



Xiaomi Smart Doorbell

MQTT or HTTP or Both?

QoS 0 or QoS 1 or QoS 2?

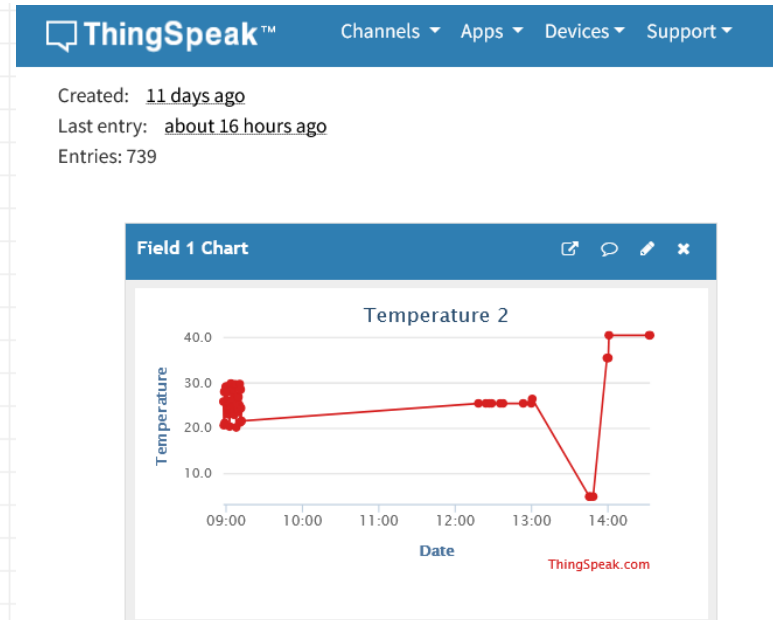
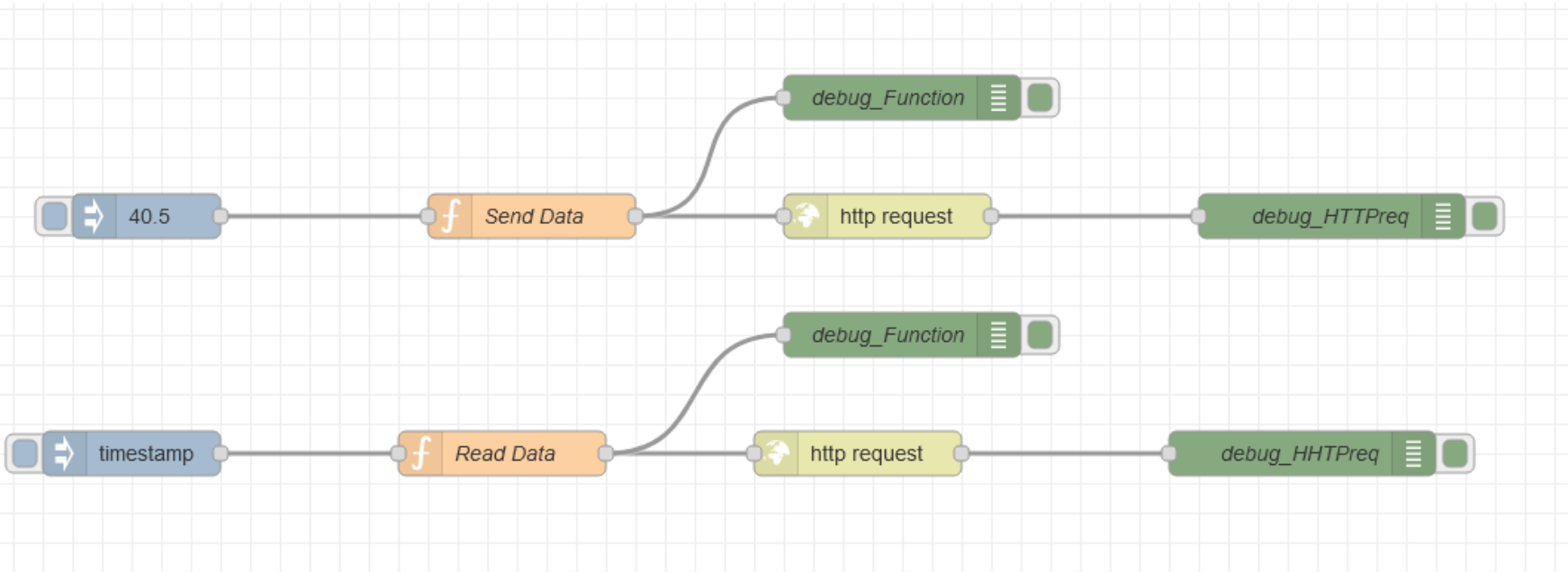
Gateway

A gateway serves as a bridge that connects two different networks or protocols, enabling data to flow between them. It can perform protocol translation, data format conversion, and handle communication tasks. In our scenario, here's how a gateway could fit into the various components:

1. **ThingSpeak:** ThingSpeak is a cloud platform for IoT applications that allows users to collect, analyze, and visualize data from their devices. It can act as a cloud gateway for data collected from various edge devices.
2. **Node-RED:** Node-RED is a flow-based development tool for visual programming. It can function as a local gateway that connects different inputs and outputs, processing data from devices (like sensors) and sending it to cloud services (like ThingSpeak) or other endpoints.
3. **Mosquitto:** Mosquitto is an open-source MQTT broker that facilitates communication in IoT applications. It acts as a gateway for MQTT messages, allowing devices to publish and subscribe to topics. In this role, it enables communication between devices and cloud services or between devices themselves.
4. **MQTT (E2C or C2E):** MQTT (Message Queuing Telemetry Transport) is a lightweight messaging protocol optimized for low-bandwidth and high-latency networks. It can facilitate Edge-to-Cloud (E2C) communication, where devices send data to cloud platforms, or Cloud-to-Edge (C2E) communication, where cloud services send commands or data back to edge devices.
5. **Sample Scripts:** Sample scripts could act as gateways by demonstrating how to implement communication between these systems. They might show how to send data from Node-RED to ThingSpeak or how to receive data from ThingSpeak.
6. **HTTP Communication:** HTTP communication paths between Node-RED and ThingSpeak represent an interface for data exchange. Node-RED can act as a gateway, making HTTP requests to ThingSpeak to send or retrieve data.
7. **MQTT Communication:** Similar to HTTP, MQTT communication paths represent an interface for sending and receiving messages. Node-RED can serve as a gateway to publish messages to Mosquitto or subscribe to messages from it, facilitating communication between devices and the cloud.

HTTP Communication

(NodeRED to ThingSpeak) vs (ThingSpeak to NodeRED)



MQTT Communication

(NodeRED to ThingSpeak) vs (ThingSpeak to NodeRED)

